```
import components.set.Set;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
/**
 * JUnit test fixture for {@code Set<String>}'s constructor and kernel methods.
 *
 * @author Put your name here
 *
 */
public abstract class SetTest {

    /**
     * Invokes the appropriate {@code Set} constructor and returns the result.
     *
     * @return the new set
     * @ensures constructorTest = {}
     */
    protected abstract Set<String> constructorTest();

    /**
     * Invokes the appropriate {@code Set} constructor and returns the result.
     *
     * @return the new set
     * @ensures constructorRef = {}
     */
    protected abstract Set<String> constructorRef();

    /**
     * Creates and returns a {@code Set<String>} of the implementation under
     * test type with the given entries.
     *
     * @param args
     *            the entries for the set
     * @return the constructed set
     * @requires [every entry in args is unique]
     * @ensures createFromArgsTest = [entries in args]
     */
    private Set<String> createFromArgsTest(String... args) {
        Set<String> set = this.constructorTest();
        for (String s : args) {
            assert !set.contains(
                    s) : "Violation of: every entry in args is unique";
            set.add(s);
        }
        return set;
    }

    /**
     * Creates and returns a {@code Set<String>} of the reference implementation
     * type with the given entries.
     *
     * @param args
     *            the entries for the set
     * @return the constructed set
     * @requires [every entry in args is unique]
     * @ensures createFromArgsRef = [entries in args]
     */
    private Set<String> createFromArgsRef(String... args) {
        Set<String> set = this.constructorRef();
        for (String s : args) {
            assert !set.contains(
                    s) : "Violation of: every entry in args is unique";
            set.add(s);
        }
        return set;
    }

    // TODO - add test cases for constructor, add, remove, removeAny, contains, and size
    @Test
```

```
    public final void testConstructor() {
        Set<String> s = this.constructorTest();
        Set<String> sExpected = this.constructorRef();

        assertEquals(sExpected, s);
    }

    @Test
    public final void testAddEmpty() {
        Set<String> s = this.createFromArgsTest();
        Set<String> sExpected = this.createFromArgsRef("a");

        s.add("a");

        assertEquals(sExpected, s);
    }

    @Test
    public final void testAddNonEmpty() {
        Set<String> s = this.createFromArgsTest("a", "b");
        Set<String> sExpected = this.createFromArgsRef("a", "b", "c");

        s.add("c");

        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveOneElement() {
        Set<String> s = this.createFromArgsTest("a");
        Set<String> sExpected = this.createFromArgsRef();

        String removed = s.remove("a");

        assertEquals("a", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveFirstOfSeveral() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");
        Set<String> sExpected = this.createFromArgsRef("b", "c");

        String removed = s.remove("a");

        assertEquals("a", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveMiddleOfSeveral() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");
        Set<String> sExpected = this.createFromArgsRef("a", "c");

        String removed = s.remove("b");

        assertEquals("b", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveLastOfSeveral() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");
        Set<String> sExpected = this.createFromArgsRef("a", "b");

        String removed = s.remove("c");

        assertEquals("c", removed);
        assertEquals(sExpected, s);
```

```
    }

    @Test
    public final void testRemoveAnyOneElement() {
        Set<String> s = this.createFromArgsTest("a");
        Set<String> sExpected = this.createFromArgsRef();

        String removed = s.removeAny();

        assertEquals("a", removed);
        assertEquals(sExpected, s);
    }

    @Test
    public final void testRemoveAnySeveral() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");

        String removed = s.removeAny();

        assertEquals(false, s.contains(removed));
        assertEquals(2, s.size());
    }

    @Test
    public final void testContainsEmptyFalse() {
        Set<String> s = this.createFromArgsTest();

        assertEquals(false, s.contains("a"));
    }

    @Test
    public final void testContainsTrue() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");

        assertEquals(true, s.contains("b"));
    }

    @Test
    public final void testContainsFalseNonEmpty() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");

        assertEquals(false, s.contains("d"));
    }

    @Test
    public final void testSizeEmpty() {
        Set<String> s = this.createFromArgsTest();

        assertEquals(0, s.size());
    }

    @Test
    public final void testSizeOne() {
        Set<String> s = this.createFromArgsTest("a");

        assertEquals(1, s.size());
    }

    @Test
    public final void testSizeSeveral() {
        Set<String> s = this.createFromArgsTest("a", "b", "c");

        assertEquals(3, s.size());
    }
}
```